



SEED order entry

Contents

0.1	Revision History	3
1	Overview	5
1.1	Optional Fields / Presence bits	5
1.2	Message Extensibility	5
1.3	Modify and Replace Behaviors	6
1.4	Pegged Orders	6
1.5	Display Price Slide Instructions	7
1.6	Market Orders	8
1.7	Hidden Orders	9
1.8	Reserve Orders	9
1.9	Auction Order Restrictions	10
1.10	Auction Band Window and Late Limit Orders	11
1.11	Post-Only Orders	12
1.12	Good-'Til-Time Orders	12
1.13	Self Match Prevention	13
1.14	Mass Cancel Behavior	14
1.15	Crossed Market Conditions	15
2	Types	17
2.1	Side	17
2.2	TimeInForce	17
2.3	OrderCapacity	18
2.4	SelfMatchScope	18
2.5	SelfMatchInstruction	19
2.6	PriceSlideInstruction	19
2.7	MassCancelScope	19
2.8	RejectReason	20
2.9	MarketParticipantId	21
2.10	CancelReason	22
2.11	LiquidityIndicator	23
2.12	RestatementReason	23
2.13	SymbolTradingState	23
2.14	ShortSaleRestrictionState	24
2.15	OperationalHaltReason	24

2.16	RegulatoryHaltReason	24
2.17	MarketHoursState	26
2.18	SessionTradingState	26
3	Messages from Member to Exchange	28
3.1	LimitOrder	28
3.1.1	LimitOrder Presence Bits	28
3.1.2	LimitOrder Optional Fields	29
3.2	MarketOrder	30
3.2.1	MarketOrder Presence Bits	30
3.2.2	MarketOrder Optional Fields	30
3.3	CancelOrder	31
3.4	ModifyOrder	31
3.4.1	ModifyOrder Presence Bits	31
3.4.2	ModifyOrder Optional Fields	31
3.5	ReplaceOrder	32
3.5.1	ReplaceOrder Presence Bits	32
3.5.2	ReplaceOrder Optional Fields	33
3.6	MassCancel	33
3.6.1	MassCancel Presence Bits	34
3.6.2	MassCancel Optional Fields	34
4	Messages from Exchange to Member	35
4.1	TradingSessionStatus	35
4.1.1	TradingSessionStatus Presence Bits	35
4.1.2	TradingSessionStatus Optional Fields	35
4.2	DefineSymbol	35
4.3	SymbolStatus	36
4.3.1	SymbolStatus Presence Bits	36
4.3.2	SymbolStatus Optional Fields	37
4.4	LimitOrderAccepted	37
4.4.1	LimitOrderAccepted Presence Bits	38
4.4.2	LimitOrderAccepted Optional Fields	39
4.5	LimitOrderRejected	39
4.5.1	LimitOrderRejected Presence Bits	40
4.5.2	LimitOrderRejected Optional Fields	41
4.6	MarketOrderAccepted	42
4.6.1	MarketOrderAccepted Presence Bits	42
4.6.2	MarketOrderAccepted Optional Fields	42
4.7	MarketOrderRejected	43
4.7.1	MarketOrderRejected Presence Bits	44
4.7.2	MarketOrderRejected Optional Fields	44
4.8	OrderCanceled	44
4.9	CancelRejected	45
4.10	OrderModified	45
4.10.1	OrderModified Presence Bits	45
4.10.2	OrderModified Optional Fields	45
4.11	ModifyRejected	46
4.11.1	ModifyRejected Presence Bits	46
4.11.2	ModifyRejected Optional Fields	46
4.12	OrderReplaced	47
4.12.1	OrderReplaced Presence Bits	48

4.12.2 OrderReplaced Optional Fields	48
4.13 ReplaceRejected	49
4.13.1 ReplaceRejected Presence Bits	50
4.13.2 ReplaceRejected Optional Fields	51
4.14 OrderExecuted	51
4.15 OrderRestated	51
4.15.1 OrderRestated Presence Bits	52
4.15.2 OrderRestated Optional Fields	52
4.16 SelfMatchPrevented	52
4.17 MassCancelAccepted	53
4.17.1 MassCancelAccepted Presence Bits	53
4.17.2 MassCancelAccepted Optional Fields	54
4.18 MassCancelRejected	54
4.18.1 MassCancelRejected Presence Bits	55
4.18.2 MassCancelRejected Optional Fields	55
4.19 MassCancelResult	55

0.1 Revision History

Date	Version	Notes
July 10, 2025	0.1	Initial version. Known gaps: Risk Controls, Liquidity code and Symbology.
September 09, 2025	0.6.draft	Alpha release. Refined message layout, identifiers and enumerations in line with early internal feedback.
September 30, 2025	0.7.draft	Increased Presence Bits storage.
October 01, 2025	0.8.draft	Replace bitfield storage increased, included isPostOnly.
October 20, 2025	0.9.draft	Extended RejectReason enumeration.
October 28, 2025	0.10.draft	Updated support contact email. Extended CancelReason and RejectReason enumerations.
November 04, 2025	0.11.draft	Updated TimelnForce documentation.
November 17, 2025	0.12.draft	Mass Cancel related enumerations.
November 21, 2025	0.13.draft	Added support for Self Match Prevention.
December 25, 2025	0.14.draft	Added validation requiring CIOrdId to increase with each order.
January 05, 2026	0.15.draft	Added LiquidityIndicator enum to OrderExecuted and SelfMatchPrevented.
January 16, 2026	0.16.draft	Added SelfMatchScope, SelfMatchInstruction, PriceSlideInstruction, and ReferencePriceTarget to Replace requests.
January 20, 2026	0.17.draft	Added support for Regulation SHO and extended related enumerations.
February 12, 2026	0.18.draft	Added SEC Rule 608 Limit Up-Limit Down (LULD) support including rejection and cancellation reasons.
March 26, 2026	0.19.draft	Added minimum price variation (tick size) rejection reasons.
April 06, 2026	0.20.draft	Updated regulatory halt reasons and short sale circuit breaker stages.
May 08, 2026	0.21.draft	Added listingMarket field to DefineSymbol.
May 09, 2026	0.22.draft	Added CancelReason for orders canceled when a client disconnects from a port with cancel-on-disconnect instructions.
May 10, 2026	0.23.draft	Added MassCancel bitfields to allow auction orders to be excluded.
May 15, 2026	0.24.draft	Added auction-related TimelnForce, CancelReason, RejectReason, RestatementReason, and LiquidityIndicator values.
June 08, 2026	0.25.draft	Added a cancellation reason for pegged orders collared by their limit price.
June 08, 2026	0.26.draft	Added a cancellation reason for odd-lot orders that cannot be executed.
June 08, 2026	0.27.draft	Clarified that selected fields may take new values in future releases.
June 08, 2026	0.28.draft	Documented how messages may be extended in future releases.

Date	Version	Notes
June 12, 2026	1.0	TXSE Specification for Trading.

1 Overview

The **SEED** protocol is the high-performance order entry interface designed to provide market participants with low-latency access to Texas Stock Exchange (TXSE) equity markets. SEED combines the best practices from proven exchange technologies with innovative enhancements specifically tailored to meet the evolving demands of modern electronic trading.

Session and Framing

SEED does not include a framing or session protocol. This function is performed by a higher-level protocol such as RAKE TCP.

1.1 Optional Fields / Presence bits

Presence bits allow messages to include only the optional fields actually needed. Each bit indicates whether an optional field is present, minimizing bandwidth usage while maintaining message flexibility.

Message Structure

When optional fields are present, messages follow this layout:

[Message Type] [Presence Bits] [Fixed Fields] [Optional Field 1] [Optional Field 2]...[Optional Field N]

The presence bits appear immediately after the message type identifier and before the fixed fields. Optional fields are appended in bit order after all fixed fields.

Example: TradingSessionStatus with RegulatoryHaltReason

Message Type: 'i' (0x69)
 Presence Bits: 0x02 (bit 1 set)
 Fixed Fields: [transactTime] [marketHoursState] [tradingState]
 Optional Fields: [regulatoryHaltReason (1 byte)]

In this example, presence bit 0x02 indicates that `regulatoryHaltReason` (bit 1) is included, while `operationalHaltReason` (bit 0) is omitted. Optional fields are always in the same order as their corresponding bit positions.

1.2 Message Extensibility

How the Exchange extends a message in a future release depends on whether that message carries optional fields via presence bits. In every case, Members should read the fields they recognize and ignore anything beyond them, rather than relying on a fixed message byte length.

Messages with only fixed fields

The Exchange may append new fields to the end of the message. Because each message is length-delimited by its framing, a Member running against an earlier specification reads the fields it recognizes and ignores the additional trailing bytes. For example, a future release could append a field to the end of `DefineSymbol`, which carries only fixed fields; an existing Member continues to read the fields it knows and ignores the rest.

Messages with optional fields via presence bits

The Exchange extends these by introducing additional optional fields, each assigned a presence bit that is currently unused; when present, a new optional field follows the existing optional fields. A Member running

against an earlier specification should ignore any presence bit it does not recognize, along with the optional field that bit introduces. For example, SymbolStatus today uses presence bits 0 and 1 (operationalHaltReason and regulatoryHaltReason) and leaves bits 2 through 7 available; a future release could add an optional field on bit 2, which an existing Member would not recognize and would ignore.

These additions leave the size and position of every existing field unchanged and are not a breaking change. The Exchange will not insert a field among existing fields, reorder fields or presence bits, or widen a message's presence-bit field without a new protocol version. In particular, a presence-bit field with no unused bits remaining cannot accept further optional fields except under a new protocol version.

1.3 Modify and Replace Behaviors

Requested Quantity

Modifies will not lose priority and so will only accept quantities which remain the same or decrease relative to the resting order. Replaces will always lose priority and so can accept quantities which remain the same, decrease or increase relative to the resting order.

Executed Quantity

When processing modify or replace requests, the Exchange accounts for executed quantity in the requested quantity to determine the order's leaves quantity.

If the requested quantity is less than or equal to the already executed quantity, the order's remaining leaves quantity is set to zero and the order closes automatically. No rejection or cancellation messages are generated in this scenario.

Example:

Original order: Buy 1,000 shares
 Executed: 600 shares (400 remaining)
 Modify request: Change order to 500 shares
 Modify accepted: Order quantity = 600, Leaves quantity = 0

1.4 Pegged Orders

Pegged orders automatically price relative to the National Best Bid and Offer (NBBO) using the referencePriceTarget field, specified in basis points relative to the NBBO spread.

Pricing Calculation

$$P_{\text{buy}} = \text{NBB} + \left(\frac{\text{referencePriceTarget}}{10,000} \times (\text{NBO} - \text{NBB}) \right) \quad (1)$$

$$P_{\text{sell}} = \text{NBO} - \left(\frac{\text{referencePriceTarget}}{10,000} \times (\text{NBO} - \text{NBB}) \right) \quad (2)$$

Examples

BUY Order:

referencePriceTarget: 2,500 (25%)
 NBB: \$10.00, NBO: \$11.00 (spread: \$1.00)
 Order Price: \$10.00 + (2,500/10,000 × \$1.00) = \$10.25

SELL Order:

referencePriceTarget: 7,000 (70%)
 NBB: \$10.00, NBO: \$11.00 (spread: \$1.00)
 Order Price: $\$11.00 - (7,000/10,000 \times \$1.00) = \$10.30$

Common Peg Types

Standard peg types can be implemented using specific referencePriceTarget values:

- **PRIMARY Peg:** referencePriceTarget = 0 (0%) - Orders price at NBB (buy) or NBO (sell)
- **MID Peg:** referencePriceTarget = 5,000 (50%) - Orders price at NBBO midpoint
- **MARKET Peg:** referencePriceTarget = 10,000 (100%) - Orders price at NBO (buy) or NBB (sell)

Execution Restrictions

Pegged orders will not execute in locked, crossed, one-sided, or no-sided markets where normal NBBO pricing relationships do not exist.

Priority and Price Rounding

When different referencePriceTarget values calculate to the same final price due to rounding, arrival time determines priority.

Narrow Spread Example: Both orders price at \$10.02, so if the 70% order arrived first, it has priority.

NBB: \$10.00, NBO: \$10.03, Spread: \$0.03
 70% BUY: $\$10.00 + (0.70 \times \$0.03) = \$10.021 \rightarrow \10.02
 90% BUY: $\$10.00 + (0.90 \times \$0.03) = \$10.027 \rightarrow \10.02

Wide Spread Example: Different final prices result in standard price-time priority as 90% BUY has better price.

NBB: \$10.00, NBO: \$10.10, Spread: \$0.10
 70% BUY: $\$10.00 + (0.70 \times \$0.10) = \$10.07$
 90% BUY: $\$10.00 + (0.90 \times \$0.10) = \$10.09$

Midpoint Order with a 50% referencePriceTarget will trade at midpoint of the NBBO, which may result in a sub-penny execution price for stocks over \$1.00. Midpoint executions might also apply to other peg targets depending on the spread at the time of execution. For example, for a stock with an NBBO greater than \$1.00 and a spread of \$0.01, all referencePriceTargets greater than or equal to 50% and less than 100% are all effectively midpoint pegs, until the spread changes.

Sub \$1.00 stocks The same peg math applies to stocks under \$1.00, but valid peg prices are floored to an MPV of \$0.0001

Peg Reference Price Target Rounding While the Member is free to enter any value for the referencePriceTarget, the Exchange may floor the peg target to a minimum variation for a given stock. For example, the Exchange may determine that for a specific stock, only increments of 10% are economically meaningful. If the Member enters a referencePriceTarget of 23%, the Exchange may floor it to 20%. The Member will see the result on the order acknowledgment. The Exchange reserves the right to change the minimum peg target increment at any time for any stock. Values of 0%, 50% and 100% will never be altered.

1.5 Display Price Slide Instructions

Display Price Slide Instructions automatically adjust order display prices to comply with Regulation NMS Rule 610(d), preventing locked or crossed markets. When an order would lock (match) or cross (trade through) the

Reg NMS markets, the Exchange will reprice the displayed price while preserving the original limit price for execution.

Price Slide Behavior

When sliding occurs:

- **Internal ranking:** Order is ranked at the locking price for priority purposes
- **Market display:** Order is displayed one tick away
- **Original limit price:** Retained for execution purposes

Message Flow

Price sliding generates the following message sequence:

- The order acknowledgment reports the order with its price field set to the original limit price
- A restatement message reports the ranked price, carrying the repricing restatement reason, when sliding occurs

Examples

Locking Scenario:

Market: AAPL \$150.00 x \$150.01

Incoming order: Buy 1000 AAPL @ \$150.01 (would lock the offer)

- NO_PRICE_SLIDE: Order canceled
- SINGLE_PRICE_SLIDE_ON_LOCK_AND_CROSS: Ranked at \$150.01, Displayed at \$150.00
- MULTIPLE_PRICE_SLIDES_ON_LOCK_AND_CROSS: Ranked at \$150.01, Displayed at \$150.00

Crossing Scenario:

Market: AAPL \$150.00 x \$150.01

Incoming order: Buy 1000 AAPL @ \$150.02 (would cross the offer)

- NO_PRICE_SLIDE: Order canceled
- SINGLE_PRICE_SLIDE_ON_LOCK_AND_CROSS: Ranked at \$150.01, Displayed at \$150.00
- MULTIPLE_PRICE_SLIDES_ON_LOCK_AND_CROSS: Ranked at \$150.01, Displayed at \$150.00
- SINGLE_PRICE_SLIDE_LOCK_ONLY: Order canceled (crossing not allowed)
- MULTIPLE_PRICE_SLIDES_LOCK_ONLY: Order canceled (crossing not allowed)

Priority Impact

Each reprice creates a new timestamp and the order loses time priority at its new price level. Orders retain relative priority among other repriced orders based on original entry time.

1.6 Market Orders

Market orders provide a convenient way to execute immediate trades without specifying a price. Internally, they are handled as limit orders with a 100% peg to the reference price, ensuring aggressive pricing (maximum price

for buy orders, minimum price for sell orders). Market orders are configured with no resting capability and will immediately cancel any unfilled portion after execution attempts, guaranteeing they never rest on the book.

1.7 Hidden Orders

Hidden orders do not have Price Slide Instructions; however, the Exchange will collar the limit price. If at order entry the order would cross the NBO (for BUY) or NBB (for SELL), the Exchange will adjust the order's limit price to the locking price. This adjustment happens only once at time of entry and the order is **NOT** repriced subsequently.

Example

Market: AAPL \$150.00 x \$150.01
 Incoming order: Buy Hidden 1000 AAPL @ \$150.10 (would cross the offer)
 Order accepted: Buy Hidden 1000 AAPL @ \$150.01 (price adjusted to lock)

Note: The order acknowledgment message will reflect the adjusted limit price.

1.8 Reserve Orders

Reserve orders utilize a "Reserve Quantity" - the non-displayed portion of an order while a portion is displayed on the TXSE Book. Both the displayed portion and the Reserve Quantity are available for execution.

Basic Behavior

Reserve orders are created by setting `isHidden = true` and specifying a `maxFloorQty` that is less than the total `orderQty`. The Exchange displays a portion while keeping the remaining quantity (Reserve Quantity) hidden.

Replenishment occurs when the displayed portion is reduced to less than a Round Lot (typically 100 shares). Each replenishment creates a new `OrderId` while preserving the original `ClOrdId`.

Replenishment Types

- **Fixed Replenishment:** Default
 - Quantity: Replenishes to `maxFloorQty`
 - Time: Immediate replenishment
- **Random Quantity Replenishment:** Replenishment quantities are randomly selected within a range in Round Lot increments only
 - Range: $(\text{maxFloorQty} - \text{maxReplenishQtyRange})$ to $(\text{maxFloorQty} + \text{maxReplenishQtyRange})$
 - Example: `MaxFloor = 500`, `Range = 200` → Possible quantities: [300, 400, 500, 600, 700] shares
- **Random Time Replenishment:** Replenishment occurs after a random delay
 - Range: 0 to `maxReplenishTimeRange`
 - Example: `MaxReplenishTimeRange=500us` → Random delay between 0-500us after execution

Note: Random Quantity and Random Time can be configured together on the same order.

Active Replenishments

At any point in time, a reserve order can have:

- 0 replenishments: During time delay periods (Random Time configuration) or transient system states due to non-atomic replenishment processing
- 1 replenishment: Normal use case with active displayed quantity
- 2 replenishments: When the first replenishment falls below Round Lot size, triggering a second replenishment while the first may still have displayed leaves quantity

Order Restatement Messages

Members receive order restatement messages for each replenishment. The restatement will indicate:

- New replenishment OrderId
- Restatement reason: RESERVE_REPLENISHED
- New displayed quantity

Key Technical Details

- Each replenishment creates a new OrderId but preserves the original ClOrdId
- Exchange obfuscates OrderIds on data feeds for replenishment orders
- Reserve Quantity remains fully executable during replenishment delays
- Random replenishment quantities must be in Round Lot increments
- If remainder is less than replenishment amount, entire remainder is displayed
- Order management operations use the original ClOrdId

1.9 Auction Order Restrictions

An order submitted with an auction-only TimeInForce (AT_THE_OPEN, AT_THE_CLOSE) is rejected with reason INVALID_FOR_AUCTION_ORDER if it carries any of the following properties:

- $\text{maxFloorQty} < \text{orderQty}$ (reserve / partial-display semantics)
- peg pricing instruction
- $\text{MinQty} > 0$
- post-only
- fully hidden
- ISO routing
- `cancelAtEntryIfCrossed`
- market order (no limit price)

Each of these presupposes a continuous-book matching model that single-price multilateral auction matching does not have, so they are rejected on auction-only orders rather than silently ignored. The same instructions may be used freely on continuous-book orders.

Continuous-book orders carrying these instructions (including reserve orders) that are resting at auction time participate in the cross: the displayed portion (up to `maxFloorQty`) executes in the displayed-priority tier and the hidden portion ($\text{orderQty} - \text{maxFloorQty}$) executes in the non-displayed tier, alongside other non-displayed orders. Their volume does not appear in the auction-only band-edge share counts (see [Auction Band Window and Late Limit Orders](#)).

1.10 Auction Band Window and Late Limit Orders

Band Window Definitions

Opening Band Window: 9:28:00 ET until the Opening Auction at 9:30:00 ET. The following order types become non-cancelable at the start of this window: AT_THE_OPEN orders and RHO orders. The Participation Band published in the AuctionBandWindow message is dynamic and may shift on each 5-second dissemination cycle; the final AuctionBandWindow is published 5 seconds before the cross. The band used to adjust late-order prices is the band in effect at the auction print, which is not itself disseminated.

Closing Band Window: 3:58:00 ET until the Closing Auction at 4:00:00 ET. The following order types become non-cancelable at the start of this window: AT_THE_CLOSE orders. The Participation Band published in the AuctionBandWindow message is dynamic and may shift on each 5-second dissemination cycle; the final AuctionBandWindow is published 5 seconds before the cross. The band used to adjust late-order prices is the band in effect at the auction print, which is not itself disseminated.

Band Window Eligibility by TimeInForce

Band Window constraints (non-cancelability and Participation Band price adjustment) apply only to orders with auction-eligible TIFs:

- AT_THE_OPEN orders are auction-classed by TIF. They become non-cancelable at the start of the Opening Band Window (9:28:00 ET).
- AT_THE_CLOSE orders are auction-classed by TIF. They become non-cancelable at the start of the Closing Band Window (3:58:00 ET).
- RHO orders on TXSE-primary-listed symbols become non-cancelable at the start of the Opening Band Window (9:28:00 ET) and are subject to Opening Band Window price adjustment. After the Opening Auction prints, any unfilled shares enter the continuous book in entry-time order.

Orders entered with TIF AT_THE_OPEN during 9:28:00–9:30:00 ET are referred to as **LOO.L** (Late Limit-On-Open) and are subject to Participation Band price adjustment at the auction print. Orders entered with TIF AT_THE_CLOSE during 3:58:00–4:00:00 ET are referred to as **LOC.L** (Late Limit-On-Close) and are subject to Participation Band price adjustment at the auction print. Regular AT_THE_OPEN / AT_THE_CLOSE orders entered before the Band Window are auction-eligible but not price-adjusted; they participate at their entered limit price.

Orders with any other TIF (DAY, SYS, IOC, GTT) are not subject to the Band Window (no non-cancelability and no price adjustment). They route to the continuous book; if resting at a cross they participate in the auction at the clearing price as continuous-book liquidity (their volume does not appear in the auction-only band-edge share counts — see below).

Late Limit Auction Order Behavior with Participation Bands

During the Band Window, the share counts published every 5 seconds in AuctionBandWindow show eligible volume at each band edge. A late limit auction order with a limit price more aggressive than the Upper Band (for bids) or the Lower Band (for offers) is counted at the corresponding band edge in these published share counts. As the band is recalculated each cycle, an order's position relative to the band may change, and its contribution to the published share counts updates accordingly; this is evaluated fresh each cycle and is not carried forward — only the order's late designation persists.

At the auction print, the order's price is adjusted to be equal to the corresponding band edge — a buy above the Upper Band is adjusted to the Upper Band, a sell below the Lower Band is adjusted to the Lower Band — and the order may participate in the auction at the adjusted price.

Band-edge share counts are auction-only: they reflect AT_THE_OPEN, AT_THE_CLOSE, LOO, LOC, LOO.L, LOC.L, and RHO (during the Opening Band Window). Resting continuous-book orders (DAY, SYS, IOC, GTT,

peg, hidden, and reserve) participate in the auction at the clearing price, but their volume does not appear in the band-edge share counts.

The excess auction side is not republished in the AuctionBandWindow message during the Band Window. The Exchange publishes the buy and sell shares eligible at each band edge; firms can derive the prevailing excess side from those share counts. The earlier AuctionPreamble cycle, which streams during the Accumulation Period, carries excessAuctionSide explicitly.

Worked examples (all share the same band, Lower \$99.80, Upper \$100.20):

1. **Limit inside the band.** A buy late limit auction order at \$100.10 participates at its entered limit; no price adjustment.
2. **Buy above the Upper Band.** A buy late limit auction order at \$101.00 has its price adjusted to the Upper Band (\$100.20) at the print and participates at that price. If the auction clears at \$100.05, the buy fills at \$100.05.
3. **Sell below the Lower Band.** A sell late limit auction order at \$99.50 has its price adjusted to the Lower Band (\$99.80) at the print and participates at that price. If the auction clears at \$100.10, the sell fills at \$100.10.

1.11 Post-Only Orders

Post-Only orders are designed to add liquidity to the order book without removing existing liquidity. These orders will only post to the book as resting orders and will never execute immediately against existing orders.

Behavior

When a Post-Only order is submitted:

- If marketable without Display Price Slide Instructions: Order is canceled (would immediately execute against existing orders)
- If marketable with Display Price Slide Instructions: Order may be repriced to avoid immediate execution, depending on the slide instruction
- If not marketable: Order posts to the book at the specified limit price

Key Characteristics

- Liquidity provision only: Guarantees the order will either rest on the book or be canceled
- No immediate execution: Eliminates the risk of unexpected immediate fills
- Works with Display Price Slide Instructions: Can be combined with price slide instructions to automatically adjust marketable orders instead of canceling them

1.12 Good-'Til-Time Orders

Good-'Til-Time (GTT) orders allow members to specify a custom expiration time using the `expireTime` field. Orders with a GTT `TimeInForce` will remain active on the Exchange until the specified time or the end of the trading day is reached, at which point any remaining quantity will be automatically canceled.

Minimum Expiration Time

The Exchange enforces a minimum expiration threshold of 1 millisecond from the current time. Orders submitted with an `expireTime` less than the current time plus 1 millisecond will be rejected with reason `INVALID_EXPIRE_TIME`.

Trading Session Lifetime Limit

GTT order lifetimes are limited to the current trading day and will not persist beyond the end of the After Hours Session. If a Member specifies an `expireTime` that extends beyond the end of the After Hours Session, the Exchange will accept the order but will automatically cancel any remaining quantity at the end of the After Hours Session rather than rejecting the order at entry. This ensures orders do not inadvertently remain active across trading days while providing members the convenience of not having to calculate exact session end times.

Session End Behavior

- Orders with `expireTime` before the After Hours Session end: Canceled at the specified `expireTime`
- Orders with `expireTime` after the After Hours Session end: Canceled at the After Hours Session end
- Cancel reason in both cases: `RELATED_TO_TIME_IN_FORCE`

1.13 Self Match Prevention

TXSE's Self Match Prevention functionality prevents orders from the same trading entity from matching against each other, supporting regulatory compliance and risk management requirements consistent with the TXSE Rule 11.007(d).

The system operates across configurable scopes including by Member (`BY_MEMBER`), by Market Participant Identifier (`BY_MPID`), by a combination of both Member and Member Group (`BY_MEMBER_GROUP`), or a combination of both MPID and Member Group (`BY_MPID_AND_MEMBER_GROUP`).

Self-match prevention settings may be configured at the port level as default parameters for all orders submitted through that connection. These port-level defaults can be overridden on a per-order basis through the optional `selfMatchScope` and `selfMatchInstruction` fields, providing members with both operational efficiency and granular control over their order interaction policies.

When a potential self-match is detected between orders, the Exchange applies the specified prevention instruction: cancel the incoming (aggressive) order, cancel the resting (passive) order, cancel both orders entirely, cancel only the smaller order if sizes differ or both if equal (`CANCEL_SMALLEST`), or cancel the overlapping portion while preserving the remainder of the larger order (`DECREMENT_AND_CANCEL`).

IMPORTANT: Only the aggressive order's self-match prevention instructions are considered during a match. The resting order's instructions are ignored.

SelfMatchPrevented Message

Orders affected by self-match prevention will generate a `SelfMatchPrevented` message for each order involved. This message reports:

- `execPrice`: The price at which the match would have occurred
- `execQty`: The shares that would have executed
- `canceledQty`: The number of shares decremented or canceled from the order
- `leavesQty`: The remaining quantity still active on the book after the prevention action

For `DECREMENT_AND_CANCEL` instructions, the `canceledQty` reflects only the overlapping portion that was removed, while `leavesQty` indicates any remaining shares that continue to rest on the book. For a full cancellation instruction (cancel the incoming order, the resting order, or both), the `canceledQty` will equal the remaining open quantity and `leavesQty` will be 0.

1.14 Mass Cancel Behavior

Cancel a large group of orders based on the Member Owned scope.

Member Owned Scope

- BY_MEMBER_OWNED_SENDER_COMPS: Scope mass cancels to all the ports owned by the Member
- BY_MEMBER_OWNED_MPIDS: Scope mass cancels to all the MPIDs owned by the Member

Optional Filter Fields

The optional filter fields are logically ANDed together. Each additional field narrows the number of orders affected.

The ClOrdId must be paired with SenderComp, and will cancel the single specific order from the order entry port associated with the SenderComp.

Cancelling of orders is done on a best-effort basis. It is not atomic. Each canceled order generates an individual order-cancellation message delivered to the port that originally entered that order, and to any drop copy ports following that order. For example, if port A submits a mass cancel that cancels an order entered by port B, only port B receives the cancellation message for that order. Port A receives only the mass-cancel acknowledgment and completion messages.

Examples

Mass Cancel on all Member Owned Ports

Mass cancel all orders entered on ports owned by the Member, but **NOT** orders entered on behalf of the Member via service bureau owned ports.

scope: BY_MEMBER_OWNED_SENDER_COMPS

Mass Cancel on all Member Owned MPIDs

Mass cancel all orders associated with the Member, across all member owned ports **AND** service bureau owned ports (on-behalf-of orders).

scope: BY_MEMBER_OWNED_MPIDS

Mass Cancel a specific MPID by a Member

Mass cancel all orders associated with the MPID, across all member owned ports **AND** service-bureau owned ports (on-behalf-of orders).

scope: BY_MEMBER_OWNED_MPIDS

mpid: ABCD

Mass Cancel by MPID (Service Bureau Initiated)

Allows a Service Bureau to mass cancel all orders for a specified MPID, limited exclusively to orders submitted on behalf of that MPID through Service Bureau-owned ports.

scope: BY_MEMBER_OWNED_SENDER_COMPS

mpid: ABCD

Auction Orders

Preservation of Opening- and Closing-Auction orders on disconnect or mass cancel is available on both Cancel-on-Disconnect (`CancelOnDisconnectInstruction.ENABLED_KEEP_AUCTIONS`) and explicit Mass Cancel (the `excludeAuctionOrders` flag).

Scope: Opening Auction and Closing Auction only. Halt Auction and IPO Auction orders are always canceled under Mass Cancel / Cancel-on-Disconnect, regardless of the election.

Once an auction-classed order has entered the Band Window, it is non-cancelable for the entire window (9:28:00–9:30:00 ET for opens, 3:58:00–4:00:00 ET for closes), regardless of any Mass Cancel scope or `excludeAuctionOrders` flag value. A Mass Cancel with `excludeAuctionOrders = false` submitted during the Band Window will cancel non-auction working orders for that MPID; auction orders already in the Band Window are unaffected.

RHO orders are canceled under Mass Cancel / Cancel-on-Disconnect prior to the Opening Band Window. Once an RHO order enters the Opening Band Window (9:28:00 ET), it is non-cancelable through the auction print.

1.15 Crossed Market Conditions

When a Protected Bid crosses a Protected Offer, the Exchange restricts executions to prevent trades at unreasonable prices, per Regulation NMS Rule 610(e).

Execution Price Restrictions During Crossed Markets

Per Rule 11.009(a)(2), during crossed markets the Exchange will not execute:

- Buy orders at prices more than 0.5% (or \$0.05, whichever is greater) above the lowest Protected Offer
- Sell orders at prices more than 0.5% (or \$0.05, whichever is greater) below the highest Protected Bid

Cancel on Crossed Market Instruction

Members may include a `cancelAtEntryIfCrossed` instruction on orders. When set, the Exchange will immediately cancel the incoming order if a Protected Bid is crossing a Protected Offer at time of entry, rather than attempting execution within the price restrictions.

Order Handling During Crossed Markets

- Limit Orders will follow Display Price Slide Instructions. Refer to [documentation](#) for details.
- Market Orders will not execute when a locked or crossed market exists.
- Pegged Orders will not execute when a locked or crossed market exists.

Examples

Crossed Market - Buy Order Execution Cap:

NBBO: \$10.05 x \$10.03 (crossed)

Max Buy Execution Price: $\$10.03 + \max(\$0.05, \$10.03 \times 0.005) = \10.08

Incoming: Buy 1000 @ \$10.10

Result: Can execute up to \$10.08; orders/liquidity above \$10.08 unavailable

Crossed Market - With Cancel Instruction:

NBBO: \$10.05 x \$10.03 (crossed)

Incoming: Buy 1000 @ \$10.10 with `cancelAtEntryIfCrossed`

Result: Order immediately canceled (no execution attempt)

High-Priced Stock Example:

NBBO: \$1,005 x \$1,000 (crossed)

Max Buy Price: $\$1,000 + \max(\$0.05, \$5.00) = \$1,005$

Incoming: Buy 100 @ \$1,010 with `cancelAtEntryIfCrossed =false`

Result: Can execute up to \$1,005; cannot access liquidity above \$1,005

ISO Exception

Orders marked as Intermarket Sweep Orders (ISO) may execute beyond these price restrictions, as the sender assumes responsibility for simultaneously routing orders to protected quotations.

2 Types

- This document uses the terms Byte, Short, Int, Long to refer to numeric values that are 8,16,32,64 bit long respectively. All are two's complement signed little-endian encoded.

Type	Byte Length	MAX_VALUE
Byte	1	$2^7 - 1 = 127$
Short	2	$2^{15} - 1 = 32,767$
Int	4	$2^{31} - 1 = 2,147,483,647$
Long	8	$2^{63} - 1 = 9,223,372,036,854,775,807$

- Strings are all ASCII, fixed-length. Strings shorter than the fixed-length should be left justified, by right padded with spaces (ASCII 0x20) to the full string length
- Enums are encoded as a single Byte unless they appear in a bitfield, then they use the number of bits specified.
- TimeStamps are nanoseconds since the Unix epoch in UTC time zone, encapsulated by a Long
- Prices are a Long with an implied scale of 100,000,000. Each unit is \$0.00000001
- Durations are represented as nanoseconds using a Long value
- Client-specified order identifier represented as a Long value that must be strictly greater than previously used values. Sending a CIOId that is less than or equal to a previously used value will result in a DUPLICATE_CLIENT_ORDER_ID rejection.

2.1 Side

Determines direction of a trading action, including short-sell indicators.

Enum Constant	Value	Notes
BUY	0	Order is a buy order.
LONG_SELL	1	Sell order of existing inventory.
SHORT_SELL	2	Sell order of borrowed shares.
SHORT_EXEMPT	3	Sell order of borrowed shares exempt from certain regulations of Regulation SHO.

2.2 TimeInForce

Determines time when an order goes live and when it will be canceled. The Exchange has three trading sessions: Early Session, Regular Session and After Hours Session. The Regular Session is the "normal" trading day, typically between 9:30 ET and 16:00 ET. The Early and After Hours Sessions are bounded by the Regular Session and the Exchange's start and end times.

Enum Constant	Value	Notes
SYS	1	Accepted during any session. Live immediately upon entry. Canceled at the end of the After Hours Session.
IOC	2	Similar to SYS, but canceled immediately, after trading with resting liquidity on the Exchange.
GTT	3	Similar to SYS, but canceled after the requested time has passed or the end of the After Hours Session, whichever comes first. See the order's expireTime field. For additional information, refer to GTT documentation .

Enum Constant	Value	Notes
DAY	4	Accepted during the Early and Regular Sessions; rejected during the After Hours Session. Live immediately upon entry. On TXSE-primary-listed symbols, resting DAY orders participate in the Closing Auction as continuous-book orders. Any unfilled shares are canceled at the conclusion of the Closing Auction. On non-TXSE-primary-listed symbols, resting DAY orders are canceled at the end of the Regular Session.
RHO	5	Regular-Hours-Only limit order. Accepted during the Early Session for TXSE-primary-listed symbols only; accepted during the Regular Session for any symbol. Rejected during the After Hours Session. For TXSE-primary-listed symbols entered before the Regular Session, the order is queued and is eligible to participate in the Opening Auction. The order becomes non-cancelable at the start of the Opening Band Window (9:28:00 ET). After the Opening Auction print, any unfilled shares enter the continuous book in entry-time order. On TXSE-primary-listed symbols, resting RHO orders participate in the Closing Auction as continuous-book orders. Any unfilled shares are canceled at the conclusion of the Closing Auction. On non-TXSE-primary-listed symbols, resting RHO orders are canceled at the end of the Regular Session.
AT_THE_OPEN	6	Auction-only TIF for the Opening Auction cross. Order participates only in the Opening cross — no continuous-book matching. Accepted from the start of the Early Session until the cross fires. Order becomes non-cancelable at the start of the Opening Band Window. Any unfilled residual is canceled after the cross. Rejected during the Regular Session, After Hours Session, and for non-TXSE-primary-listed symbols.
AT_THE_CLOSE	7	Auction-only TIF for the Closing Auction cross. Order participates only in the Closing cross — no continuous-book matching. Accepted from the start of the Early Session until the cross fires. Order becomes non-cancelable at the start of the Closing Band Window. Any unfilled residual is canceled after the cross. Rejected during the After Hours Session and for non-TXSE-primary-listed symbols.

2.3 OrderCapacity

Broker capacity in sending order.

Enum Constant	Value	Notes
AGENCY	1	Acting on behalf of clients.
PRINCIPAL	2	Acting on behalf of Broker-Dealer.
RISKLESS_PRINCIPAL	3	Order to offset client order.

2.4 SelfMatchScope

Determines the scope for self-match prevention. For additional information, refer to [Self Match Prevention documentation](#).

Enum Constant	Value	Notes
BY_MEMBER	0	Match prevention applies to all orders from the same Member.
BY_MPID	1	Match prevention applies to orders from the same Member with the same MPID.
BY_MEMBER_GROUP	2	Match prevention applies to orders from the same Member within the same Member Group.

Enum Constant	Value	Notes
BY_MPID_AND_MEMBER_GROUP	3	Match prevention applies to orders from the same Member with the same MPID and Member Group.

2.5 SelfMatchInstruction

Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention [documentation](#).

Enum Constant	Value	Notes
NO_SELF_MATCH_PREVENTION	0	Self-match prevention is disabled.
CANCEL_NEWEST	1	Cancels the incoming order. Keeps the resting order on the book.
CANCEL_OLDEST	2	Cancels the resting order. Keeps the incoming order on the book.
CANCEL_BOTH	3	Cancels both orders entirely regardless of size.
CANCEL_SMALLEST	4	If orders are equal size: cancels both. If different sizes: cancels the smaller order, keeps the larger one.
DECREMENT_AND_CANCEL	5	If orders are equal size: cancels both. If different sizes: cancels the overlapping portion, keeps the remainder of the larger order.

2.6 PriceSlideInstruction

For additional information, refer to Display Price Slide Instructions [documentation](#).

Enum Constant	Value	Notes
NO_PRICE_SLIDE	0	No Display-Price Slide instruction. Orders that would create locking or crossing quotations will be canceled.
SINGLE_PRICE_SLIDE_ON_LOCK_AND_CROSS	1	Single price slide applied to both locking and crossing quotations. Adjusts order price only upon entry and one additional time following an NBBO change.
MULTIPLE_PRICE_SLIDES_ON_LOCK_AND_CROSS	2	Multiple price slides applied to both locking and crossing quotations. Continuously re-ranks and re-displays orders as NBBO changes throughout the order's life.
SINGLE_PRICE_SLIDE_LOCK_ONLY	3	Single price slide applied only to locking quotations. Crossing quotations will be canceled. Note: Given this only serves locked quotations, a multiple price slide option could only slide once, thus making it redundant.

2.7 MassCancelScope

Determines the Member-Owned scope for mass cancels. For additional information, refer to Mass Cancel Behavior [documentation](#).

Enum Constant	Value	Notes
BY_MEMBER_OWNED_SENDER_COMPS	0	
BY_MEMBER_OWNED_MPIDS	1	

2.8 RejectReason

Indicates the reason that a customer action was rejected by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

Enum Constant	Value	Notes
INVALID_CLIENT_ORDER_ID	1	Client Order Id is invalid.
DUPLICATE_CLIENT_ORDER_ID	2	Client Order Id has already been used for the current session on this port.
UNKNOWN_ORIGINAL_CLIENT_ORDER_ID	3	Attempt to modify or replace a client order that was not found for the current session on this port.
NO_LONGER_ON_BOOK	4	Attempt to modify, replace, or cancel an order that has already canceled or fully executed.
INVALID_SYMBOL	5	Symbol ID is not valid for the current session.
INVALID_PRICE	6	Invalid limit price.
INVALID_ORDER_QUANTITY	7	Invalid order quantity.
INVALID_REFERENCE_PRICE_TARGET	8	Invalid reference price target related to the order attributes.
INVALID_IS_HIDDEN_FLAG	9	Invalid 'Is Hidden' flag related to the order attributes.
INVALID_ORDER_TYPE	10	Invalid order type related to the order attributes.
INVALID_SIDE	11	Invalid side.
INVALID_MAX_FLOOR_QUANTITY	12	Invalid max floor quantity related to the order attributes.
INVALID_MAX_REPLENISH_QUANTITY_RANGE	13	Invalid maximum replenish quantity range.
INVALID_MAX_REPLENISH_TIME_RANGE	14	Invalid maximum replenish time range.
INVALID_MINIMUM_QUANTITY	15	Invalid minimum quantity.
INVALID_LOCATE_REQUIRED_FLAG	16	Invalid locate-required flag related to the order attributes.
INVALID_TIME_IN_FORCE	17	Invalid time in force.
MODIFICATION_NOT_PERMITTED	18	Order modification not permitted.
INVALID_MPID	19	Invalid market participant identifier.
INVALID_SENDER_COMP	20	Invalid sender comp. Must correspond to a configured order entry gateway.
INVALID_IS_POST_ONLY_FLAG	21	Invalid 'Is Post Only' flag related to the order attributes.
INVALID_EXPIRE_TIME	22	Invalid expire time.
INVALID_FOR_TRADING_SESSION	23	Not permitted during the current trading session.
INVALID_FOR_TRADING_STATUS	24	Not permitted during the current trading status.

Enum Constant	Value	Notes
INVALID_FOR_SYMBOL_STATUS	25	Not permitted during the current symbol status.
INVALID_IS_ISO_FLAG	26	Invalid 'Is ISO' flag related to the order attributes (ISO = Intermarket Sweep Order).
TRADING_DISABLED_FOR_MPID_ON_PORT	27	Trading is disabled for the MPID on this port based on port MPID settings configuration.
TRADING_DISABLED_FOR_NON_TEST_SYMBOLS	28	Trading is restricted to test symbols only based on port settings configuration.
TRADING_DISABLED_FOR_ORDER_CAPACITY	29	Trading is disabled for the specified order capacity based on port settings configuration.
TRADING_DISABLED_FOR_ISO	30	ISO (Intermarket Sweep Order) trading is disabled based on port settings configuration.
MAXIMUM_ORDER_QUANTITY_BREACHED	31	Order quantity exceeds the maximum shares limit configured in port settings.
MAXIMUM_PRICE_BREACHED	32	Limit price exceeds the maximum acceptable price for the Exchange.
MAXIMUM_NOTIONAL_BREACHED	33	Order notional value exceeds the maximum notional limit configured in port settings.
INVALID_MASS_CANCEL_REQUEST_ID	34	Mass Cancel Request Id is invalid.
SYMBOL_ON_RESTRICTED_LIST	35	Symbol is on the restricted list for this port/MPID based on port settings configuration.
LULD_BAND_BREACHED	36	Order limit price is outside LULD (Limit Up-Limit Down) bands and port settings require rejection.
INVALID_PRICE_INCREMENT	37	Price does not conform to the minimum price variation (tick size).
AUCTION_ORDER_NOT_CANCELABLE	38	Auction-Only orders for the Open and Close cannot be cancelled, modified, or replaced once the Band Window begins.
INVALID_FOR_AUCTION_ORDER	39	Order property is incompatible with an auction-classed order.

2.9 MarketParticipantId

Market participant identifier for the primary listing market. Currently, this field is populated only when TXSE is the listing market; NOT_AVAILABLE (0x00) is used otherwise. Additional listing market values may be defined in future revisions to identify listing markets for other exchanges. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

Enum Constant	Value	Notes
NOT_AVAILABLE	0	TXSE is not the listing market for this symbol.
TXSE	70	Texas Stock Exchange, LLC. CTA CTS and UTP participant code 'F'.

2.10 CancelReason

Reason that an order was canceled. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

Enum Constant	Value	Notes
REQUESTED_BY_USER	1	Cancel due to request by client.
RELATED_TO_TIME_IN_FORCE	2	Cancel due to time in force expiration.
RELATED_TO_MIN_QTY	3	Cancel due to inability to meet minimum quantity.
REG_NMS_VIOLATION_NO_SLIDE	4	Canceled as order would violate Regulation NMS Rule 610(d) by creating a locked or crossed market, and price sliding was either not selected or not applicable.
MARKETABLE_RESERVE	5	Reserve orders will be canceled when deemed marketable at another exchange either at new order entry or replenishment.
SELF_MATCH_PREVENTION	6	Cancel due to self-match prevention instructions.
REPLENISHMENT_CANCELED_DUE_TO_RESERVE	7	Replenishment canceled related to the Reserve order being canceled or replaced.
RELATED_TO_ORDER_TYPE	8	Cancel due to order type e.g. Market Orders do not rest.
CANCELED_DUE_TO_CROSSED_MARKETS	9	Cancel due to client instruction on crossed markets.
CANCELED_DUE_TO_MASS_CANCEL_REQUEST	10	Cancel due to a mass cancel request.
EXCHANGE_LOCKED_OR_CROSSED_NO_SLIDE	11	Canceled as order would lock or cross the Exchange book, and price sliding was either not selected or not applicable.
REG_SHO_VIOLATION_NO_SLIDE	12	Canceled as order would violate Regulation SHO Rule 201 by display at or below the National Best Bid during a short sale price test restriction, and price sliding was either not selected or not applicable.
LULD_BREACHED_NO_SLIDE	13	Canceled as order would violate LULD (Limit Up-Limit Down) price bands per SEC Rule 608, and price sliding was either not selected or not applicable.
CANCELED_AS_PRICE_BREACHED_LULD	14	Canceled as the order's limit price is now outside LULD (Limit Up-Limit Down) price bands after bands narrowed.
CANCELED_ON_DISCONNECT	15	Cancel due to client disconnect from a port with cancel-on-disconnect instructions.

Enum Constant	Value	Notes
AUCTION_ORDER_POST_CROSS	16	Auction order is canceled because the auction it was eligible for has concluded. Covers two cases: (a) the cross ran but did not fully fill this order — canceled at end of the post-cross auction-cancels step; (b) the regular auction never ran (e.g. symbol was halted through the close).
PEG_REFERENCE_PRICE_BREACHES_LIMIT_PRICE	17	Pegged order canceled because its calculated reference price moved beyond the order's limit price (the limit price collared the peg).
ODD_LOT_NOT_EXECUTABLE	18	Canceled as the order is of odd lot size and could not be executed against the contra-side liquidity on the TXSE Book.

2.11 LiquidityIndicator

Liquidity indicator codes for executions. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

Enum Constant	Value	Notes
REMOVED_HIDDEN_LIQUIDITY	0	Removed hidden liquidity.
REMOVED_DISPLAYED_LIQUIDITY	1	Removed displayed liquidity.
ADDED_HIDDEN_LIQUIDITY	2	Added hidden liquidity.
ADDED_DISPLAYED_LIQUIDITY	3	Added displayed liquidity.
AUCTION	4	Execution from an auction cross.

2.12 RestatementReason

Reason for order restatement events triggered by the Exchange.

Enum Constant	Value	Notes
RESERVE_REPLENISHED	1	A new displayed slice of a reserve order has been added to the displayed book.
REPRICED	2	A reprice eligible order has been repriced to a new price.
POST_AUCTION	3	Orders staged awaiting the result of the auction with unfilled residual have been reinstated into continuous book.

2.13 SymbolTradingState

Symbol-specific trading state indicating current availability.

Enum Constant	Value	Notes
TRADING	1	Symbol is open and available for trading.
HALTED	2	Trading suspended. The cause, if available, is carried in the operationalHaltReason or regulatoryHaltReason optional fields of the accompanying Symbol-Status message.

Enum Constant	Value	Notes
RESUME_PENDING	3	Applies only when a symbol is in a non-trading state with an auction cross pending to bring it back into trading. Covers HALT reopening crosses and IPO crosses. The accompanying AuctionBandWindow message identifies the specific auction type.

2.14 ShortSaleRestrictionState

Short sale restriction state under SEC Rule 201.

Enum Constant	Value	Notes
NONE	0	No short sale restriction in effect (CTA: Space, UTP: 0).
ACTIVATED	1	Restriction activated intra-day (CTA: A, UTP: 1).
CONTINUED	2	Restriction carried over from previous trading day (CTA: C, UTP: 2).
DEACTIVATED	3	Restriction deactivated intra-day (CTA: D).
IN_EFFECT	4	Restriction in effect, status unchanged from previous close (CTA: E).

2.15 OperationalHaltReason

Operational and technical reasons for trading halts initiated by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

Enum Constant	Value	Notes
ADMINISTRATIVE	0	Administrative or technical halt initiated by the Exchange.

2.16 RegulatoryHaltReason

Regulatory reasons for trading halts and pauses. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

Enum Constant	Value	Notes
REASON_NOT_AVAILABLE	0	Reason not available or not applicable (CTA: Space, UTP: Space).
NEWS_PENDING	1	Trading halted pending release of material news (CTA: P, UTP: T1).
NEWS_DISSEMINATION	2	Trading halted for dissemination of material news (CTA: D, UTP: T2).
INFO_REQUESTED	3	Additional information requested by the listing exchange (CTA: A, UTP: T12).

Enum Constant	Value	Notes
ORDER_IMBALANCE	4	Order imbalance (CTA: I).
EXTRAORDINARY_MARKET_ACTIVITY	5	Extraordinary market activity (UTP: T6).
LULD_PAUSE	6	Limit Up-Limit Down pause (CTA: M, UTP: LUDP).
NON_COMPLIANCE	7	Non-compliance with listing standards (UTP: H4).
FILINGS_NOT_CURRENT	8	Required filings not current (UTP: H9).
SEC_SUSPENSION	9	SEC trading suspension (UTP: H10).
REGULATORY_CONCERN	10	Regulatory concern from other markets (CTA: C, UTP: H11).
SUB_PENNY_TRADING	11	Sub-penny trading violation (CTA: Y).
ETF	12	ETF-specific halt (CTA: F, UTP: T8).
IPO_NOT_TRADING	13	IPO not yet trading (UTP: IPO1).
CORPORATE_ACTION	14	Corporate action in progress (CTA: N, UTP: M1).
CIRCUIT_BREAKER_L1	15	Market-wide circuit breaker Level 1 (CTA: 1, UTP: MWC1).
CIRCUIT_BREAKER_L2	16	Market-wide circuit breaker Level 2 (CTA: 2, UTP: MWC2).
CIRCUIT_BREAKER_L3	17	Market-wide circuit breaker Level 3 (CTA: 3, UTP: MWC3).
CIRCUIT_BREAKER_CARRYOVER	18	Market-wide circuit breaker halt carried over from previous trading day (UTP: MWC0).
MERGER_EFFECTIVE	19	Merger effective (CTA: E).
NEW_SECURITY_OFFERING	20	New security offering (CTA: O).
INDICATIVE_VALUE_NOT_AVAILABLE	21	Intraday indicative value not available (CTA: V).
QUOTATION_NOT_AVAILABLE	22	Quotation not available (UTP: M2).
SINGLE_STOCK_TRADING_PAUSE	23	Single stock trading pause (UTP: T5).
OPERATIONAL	24	Non-regulatory halt due to technical or operational reasons (CTA: X, UTP: O1).
LULD_PAUSE_STRADDLE	25	Limit Up-Limit Down volatility pause - straddle condition (bid/offer crosses a band) (UTP: LUDS).
NEWS_RESUMPTION_TIMES	26	News and resumption times announced (UTP: T3).
SINGLE_STOCK_TRADING_PAUSE_QUOTING	27	Single stock trading pause (UTP: T7).

Enum Constant	Value	Notes
QUALIFICATIONS_ISSUES_RESOLVED	28	Qualifications issues reviewed/resolved; quotations/trading to resume (UTP: R4).
FILING_REQUIREMENTS_SATISFIED	29	Filing requirements satisfied/resolved; quotations/trading to resume (UTP: R9).
ISSUER_NEWS_NOT_FORTHCOMING	30	Issuer news not forthcoming; quotations/trading to resume (UTP: C3).
QUALIFICATIONS_HALT_ENDED	31	Qualifications halt ended; maintenance requirements met; resume (UTP: C4).
QUALIFICATIONS_HALT_CONCLUDED	32	Qualifications halt concluded; filings met; quotes/trades to resume (UTP: C9).
REGULATORY_AUTHORITY_HALT_CONCLUDED	33	Trade halt concluded by other regulatory authority; quotes/trades to resume (UTP: C11).
NEW_ISSUE_AVAILABLE	34	New issue available (UTP: R1).
ISSUE_AVAILABLE	35	Issue available (UTP: R2).
IPO_QUOTATION_RELEASED	36	IPO security released for quotation (UTP: IPOQ).
IPO_POSITIONING_WINDOW_EXTENSION	37	IPO security - positioning window extension (UTP: IPOE).
CIRCUIT_BREAKER_QUOTATION_RESUMPTION	38	Market-wide circuit breaker quotation resumption (UTP: MWCQ).

2.17 MarketHoursState

Current phase of the market trading day.

Enum Constant	Value	Notes
CLOSED_BEFORE_HOURS	0	Markets are closed before trading hours.
EARLY_SESSION	1	Early trading session is active (pre-market).
REGULAR_SESSION	2	Regular trading session is active.
AFTER_HOURS_SESSION	3	After-hours trading session is active (post-market).
CLOSED_AFTER_HOURS	4	Markets are closed after trading hours.

2.18 SessionTradingState

Market-wide trading state indicating the default state for all symbols (individual symbols may have additional restrictions).

Enum Constant	Value	Notes
CLOSED	0	Market session is closed - outside of trading hours or not yet opened.
TRADING	1	Market session is open and trading is active (individual symbols may still be halted/paused).

Enum Constant	Value	Notes
HALTED	2	Market-wide trading halt - all symbols halted (circuit breaker or system-wide halt).

3 Messages from Member to Exchange

Messages sent from the Member to the Exchange.

3.1 LimitOrder

Field Name	Offset	Size	Type	Notes			
messageType	0	1	Byte	'L'/0x4c/76			
presenceBits	1	4	Int	See Optional Fields			
clOrdId	5	8	Long	Client-specified order identifier that must increase with each new order.			
orderQty	13	4	Int				
limitOrderBitFields	17	4	Bit Field Name		Offset	Size	Notes
			side		0	3	Enum Side .
			isLocateRequired		3	1	True if a locate is required for short-sale, false otherwise.
			timeInForce		4	4	Enum TimeInForce .
			orderCapacity		8	3	Enum OrderCapacity .
			isIso		11	1	True for Reg NMS Inter-market sweep orders, false otherwise.
			isHidden		12	1	True if any of the order quantity is hidden, false for order quantity to be displayed.
			isPostOnly		13	1	True to only add the order to the book if it would be added as a resting order with no quantity executed.
			cancelAtEntryIfCrossed		14	1	True to cancel the order at entry if market conditions are crossed.
			reserved		15	17	Do not use – reserved for the future.
symbolId	21	2	Short				
price	23	8	Long				

3.1.1 LimitOrder Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasSelfMatchScope	0x00000001
hasSelfMatchInstruction	0x00000002
hasPriceSlideInstruction	0x00000004
hasMinQty	0x00000008
hasMaxFloorQty	0x00000010
hasMaxReplenishQtyRange	0x00000020
hasMaxReplenishTimeRange	0x00000040
hasReferencePriceTarget	0x00000080
hasExpireTime	0x00000100
hasUserData	0x00000200
hasMpid	0x00000400
hasMemberGroup	0x00000800
hasLocateBroker	0x00001000
reserved	0x7FFFE000

3.1.2 LimitOrder Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
priceSlideInstruction	1	Enum	Enum PriceSlideInstruction . For additional information, refer to Display Price Slide Instructions documentation .
minQty	4	Int	Number of shares that must be executed to execute any shares.
maxFloorQty	4	Int	Quantity to be displayed at one time with remaining quantity not displayed on the book.
maxReplenishQtyRange	4	Int	Defines the maximum range to be used when calculating a random refresh quantity.
maxReplenishTimeRange	8	Duration	Defines the maximum range to be used when calculating a random refresh time, in nanoseconds.
referencePriceTarget	2	Short	The target relative to the reference base price in basis points of the NBBO spread.
expireTime	8	TimeStamp	Timestamp at which the order's open quantity will be canceled, valid only with GTT time in force. For additional information, refer to GTT documentation .
userData	8	Long	Pass-through field for use by clients, not used by the Exchange.
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or SenderComp.

Field Name	Size	Type	Notes
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

3.2 MarketOrder

Field Name	Offset	Size	Type	Notes			
messageType	0	1	Byte	'A'/0x41/65			
presenceBits	1	2	Short	See Optional Fields			
clOrdId	3	8	Long	Client-specified order identifier that must increase with each new order.			
orderQty	11	4	Int				
marketOrderBitFields	15	2	Bit Field Name		Offset	Size	Notes
			side		0	3	Enum Side.
			isLocateRequired		3	1	True if a locate is required for short-sale, false otherwise.
			timeInForce		4	4	Enum TimeInForce.
			orderCapacity		8	3	Enum OrderCapacity.
			reserved		11	5	Do not use – reserved for the future.
symbolId	17	2	Short				

3.2.1 MarketOrder Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasSelfMatchScope	0x0001
hasSelfMatchInstruction	0x0002
hasUserData	0x0004
hasMpid	0x0008
hasMemberGroup	0x0010
hasLocateBroker	0x0020
reserved	0xFFC0

3.2.2 MarketOrder Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .

Field Name	Size	Type	Notes
userData	8	Long	Pass-through field for use by clients, not used by the Exchange.
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or Sender-Comp.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

3.3 CancelOrder

Cancel all outstanding shares of an order.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'C'/0x43/67
origClOrdId	1	8	Long	The ClOrdId of the order to be canceled.

3.4 ModifyOrder

Request to modify an existing order without losing book priority.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'M'/0x4d/77
presenceBits	1	1	Byte	See Optional Fields
clOrdId	2	8	Long	The ClOrdId for this modify message. Must be day-unique.
origClOrdId	10	8	Long	The ClOrdId of the order to be modified.

3.4.1 ModifyOrder Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasOrderQty	0x01
hasModifyBitFields	0x02
hasLocateBroker	0x04
reserved	0xF8

3.4.2 ModifyOrder Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
orderQty	4	Int	Quantity can remain the same or decrease. Quantity cannot increase. For additional information, refer to Modify Behavior documentation .

Field Name	Size	Type	Notes		
modifyBitFields	1	Bit Field Name	Offset	Size	Notes
		side	0	3	Enum Side .
		isLocateRequired	3	1	Processed for Sell Short and Sell Short Exempt in which client affirms ability to borrow (isLocateRequired = false) or client does not affirm ability to borrow (isLocateRequired = true)
		reserved	4	4	Do not use – reserved for the future.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.		

3.5 ReplaceOrder

Replace an existing order with a new order, always losing priority.

Field Name	Offset	Size	Type	Notes		
messageType	0	1	Byte	'R'/0x52/82		
presenceBits	1	2	Short	See Optional Fields		
clOrdId	3	8	Long	The ClOrdId for this modify message. Must be day-unique.		
origClOrdId	11	8	Long	The ClOrdId of the order to be replaced.		
replaceBitFields	19	2	Bit Field Name	Offset	Size	Notes
			side	0	3	Enum Side .
			isLocateRequired	3	1	Processed for Sell Short and Sell Short Exempt in which client affirms ability to borrow (isLocateRequired = false) or client does not affirm ability to borrow (isLocateRequired = true)
			isIso	4	1	True for Reg NMS Inter-market sweep orders, false otherwise.
			isPostOnly	5	1	True to only add the order to the book if it would be added as a resting order with no quantity executed.
			cancelAtEntryIfCrossed	6	1	True to cancel the order at entry if market conditions are crossed.
			reserved	7	9	Reserved for future use.

3.5.1 ReplaceOrder Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasPrice	0x0001
hasOrderQty	0x0002
hasMaxFloorQty	0x0004
hasSelfMatchScope	0x0008
hasSelfMatchInstruction	0x0010
hasPriceSlideInstruction	0x0020
hasReferencePriceTarget	0x0040
hasLocateBroker	0x0080
reserved	0xFF00

3.5.2 ReplaceOrder Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
price	8	Long	
orderQty	4	Int	Quantity can remain the same, decrease or increase. For additional information, refer to Replace Behavior documentation .
maxFloorQty	4	Int	Quantity to be displayed at one time with remaining quantity not displayed on the book.
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
priceSlideInstruction	1	Enum	Enum PriceSlideInstruction . For additional information, refer to Display Price Slide Instructions documentation .
referencePriceTarget	2	Short	The target relative to the reference base price in basis points of the NBBO spread.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

3.6 MassCancel

Cancels all resting orders matching the specified scope and optional filters. For additional information, refer to Mass Cancel Behavior [documentation](#).

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'V'/0x56/86
presenceBits	1	1	Byte	See Optional Fields
massCancelRequestId	2	8	Long	Unique identifier assigned by the Member for this mass cancel request.

Field Name	Offset	Size	Type	Notes
scope	10	1	Enum	Enum MassCancelScope . Determines the Member-Owned scope for mass cancels. For additional information, refer to Mass Cancel Behavior documentation .
massCancelBitFields	11	1	Bit Field Name	Offset
			excludeAuctionOrders	0
			reserved	1
			Size	Notes
			1	When set, auction orders are excluded from the mass cancel scope.
			7	Do not use – reserved for the future.

3.6.1 MassCancel Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasMpid	0x01
hasSenderComp	0x02
hasMemberGroup	0x04
hasClOrdId	0x08
reserved	0xF0

3.6.2 MassCancel Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
senderComp	8	Str(8)	When present, narrows the cancel to orders entered on the specified order entry port (SenderComp).
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or SenderComp.
clOrdId	8	Long	When present, scopes the mass cancel to this specific order. SenderComp is required when ClOrdId is supplied.

4 Messages from Exchange to Member

Messages sent from the Exchange to the Member.

4.1 TradingSessionStatus

Market-wide status representing the overall state of the trading session.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'i'/0x69/105
presenceBits	1	1	Byte	See Optional Fields
transactTime	2	8	TimeStamp	
marketHoursState	10	1	Enum	Enum MarketHoursState . Current phase of the market trading day.
tradingState	11	1	Enum	Enum SessionTradingState . Market-wide trading state indicating the default state for all symbols (individual symbols may have additional restrictions).

4.1.1 TradingSessionStatus Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasOperationalHaltReason	0x01
hasRegulatoryHaltReason	0x02
reserved	0xFC

4.1.2 TradingSessionStatus Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
operationalHaltReason	1	Enum	Enum OperationalHaltReason . Operational and technical reasons for trading halts initiated by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.
regulatoryHaltReason	1	Enum	Enum RegulatoryHaltReason . Regulatory reasons for trading halts and pauses. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.2 DefineSymbol

Symbol definition containing instrument details and trading parameters for a security. A DefineSymbol will be followed by a SymbolStatus message establishing the initial trading state for the symbol. Clients must treat a

symbol as operationally HALTED while awaiting the accompanying SymbolStatus.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	's'/0x73/115
transactTime	1	8	TimeStamp	
symbolId	9	2	Short	
symbol	11	8	Str(8)	Security identifier root represented in CMS format.
suffix	19	8	Str(8)	Security identifier suffix represented in CMS format.
matchingEngineId	27	1	Byte	The matching engine to which this symbol is assigned for the trading session.
defineSymbolBitFields	28	1	Bit Field Name	Offset
			isTest	0
			reserved	1
			Size	Notes
			1	Test symbol flag.
			7	Reserved for future use.
lotSize	29	4	Int	The number of shares constituting one round lot.
listingMarket	33	1	Enum	Enum MarketParticipantId . Market participant identifier of the listing exchange. Set to TXSE when TXSE is the listing market for this symbol; NOT_AVAILABLE (0x00) otherwise.

4.3 SymbolStatus

Symbol-specific trading status representing the current state and restrictions for an individual security. A SymbolStatus is sent for every symbol, including those that open in the TRADING state. Clients must treat a symbol as operationally HALTED while awaiting the first SymbolStatus. Subsequent SymbolStatus messages are sent whenever the trading state or short-sale restriction changes.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'y'/0x79/121
presenceBits	1	1	Byte	See Optional Fields
transactTime	2	8	TimeStamp	
symbolId	10	2	Short	
tradingState	12	1	Enum	Enum SymbolTradingState . Symbol-specific trading state indicating current availability.
shortSaleRestrictionState	13	1	Enum	Enum ShortSaleRestrictionState . Short sale restriction state under SEC Rule 201.

4.3.1 SymbolStatus Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasOperationalHaltReason	0x01
hasRegulatoryHaltReason	0x02
reserved	0xFC

4.3.2 SymbolStatus Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
operationalHaltReason	1	Enum	Enum OperationalHaltReason . Operational and technical reasons for trading halts initiated by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.
regulatoryHaltReason	1	Enum	Enum RegulatoryHaltReason . Regulatory reasons for trading halts and pauses. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.4 LimitOrderAccepted

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'I'/0x49/73
presenceBits	1	4	Int	See Optional Fields
transactTime	5	8	TimeStamp	
orderId	13	8	Long	
clOrdId	21	8	Long	Client-specified order identifier that must increase with each new order.
orderQty	29	4	Int	

Field Name	Offset	Size	Type	Notes		
limitOrderBitFields	33	4	Bit Field Name	Offset	Size	Notes
			side	0	3	Enum Side .
			isLocateRequired	3	1	True if a locate is required for short-sale, false otherwise.
			timeInForce	4	4	Enum TimeInForce .
			orderCapacity	8	3	Enum OrderCapacity .
			isIso	11	1	True for Reg NMS Inter-market sweep orders, false otherwise.
			isHidden	12	1	True if any of the order quantity is hidden, false for order quantity to be displayed.
			isPostOnly	13	1	True to only add the order to the book if it would be added as a resting order with no quantity executed.
			cancelAtEntryIfCrossed	14	1	True to cancel the order at entry if market conditions are crossed.
			reserved	15	17	Do not use – reserved for the future.
symbolId	37	2	Short			
price	39	8	Long			

4.4.1 LimitOrderAccepted Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasSelfMatchScope	0x00000001
hasSelfMatchInstruction	0x00000002
hasPriceSlideInstruction	0x00000004
hasMinQty	0x00000008
hasMaxFloorQty	0x00000010
hasMaxReplenishQtyRange	0x00000020
hasMaxReplenishTimeRange	0x00000040
hasReferencePriceTarget	0x00000080
hasExpireTime	0x00000100
hasUserData	0x00000200
hasMpid	0x00000400
hasMemberGroup	0x00000800
hasLocateBroker	0x00001000
hasRankPrice	0x00002000
hasDisplayPrice	0x00004000
reserved	0x7FFF8000

4.4.2 LimitOrderAccepted Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
priceSlideInstruction	1	Enum	Enum PriceSlideInstruction . For additional information, refer to Display Price Slide Instructions documentation .
minQty	4	Int	Number of shares that must be executed to execute any shares.
maxFloorQty	4	Int	Quantity to be displayed at one time with remaining quantity not displayed on the book.
maxReplenishQtyRange	4	Int	Defines the maximum range to be used when calculating a random refresh quantity.
maxReplenishTimeRange	8	Duration	Defines the maximum range to be used when calculating a random refresh time, in nanoseconds.
referencePriceTarget	2	Short	The target relative to the reference base price in basis points of the NBBO spread.
expireTime	8	TimeStamp	Timestamp at which the order's open quantity will be canceled, valid only with GTT time in force. For additional information, refer to GTT documentation .
userData	8	Long	Pass-through field for use by clients, not used by the Exchange.
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or SenderComp.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.
rankPrice	8	Long	Optional rank price, when different from limit price.
displayPrice	8	Long	Optional display price, when different from limit price.

4.5 LimitOrderRejected

The submitted limit order has been rejected by the Exchange.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'U'/0x55/85
presenceBits	1	4	Int	See Optional Fields
transactTime	5	8	TimeStamp	

Field Name	Offset	Size	Type	Notes		
clOrdId	13	8	Long	Client-specified order identifier that must increase with each new order.		
orderQty	21	4	Int			
limitOrderBitFields	25	4	Bit Field Name	Offset	Size	Notes
			side	0	3	Enum Side .
			isLocateRequired	3	1	True if a locate is required for short-sale, false otherwise.
			timeInForce	4	4	Enum TimeInForce .
			orderCapacity	8	3	Enum OrderCapacity .
			isIso	11	1	True for Reg NMS Inter-market sweep orders, false otherwise.
			isHidden	12	1	True if any of the order quantity is hidden, false for order quantity to be displayed.
			isPostOnly	13	1	True to only add the order to the book if it would be added as a resting order with no quantity executed.
			cancelAtEntryIfCrossed	14	1	True to cancel the order at entry if market conditions are crossed.
			reserved	15	17	Do not use – reserved for the future.
symbolId	29	2	Short			
price	31	8	Long			
reason	39	1	Enum	Enum RejectReason . Indicates the reason that a customer action was rejected by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.		

4.5.1 LimitOrderRejected Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasSelfMatchScope	0x00000001
hasSelfMatchInstruction	0x00000002
hasPriceSlideInstruction	0x00000004
hasMinQty	0x00000008
hasMaxFloorQty	0x00000010
hasMaxReplenishQtyRange	0x00000020
hasMaxReplenishTimeRange	0x00000040
hasReferencePriceTarget	0x00000080
hasExpireTime	0x00000100
hasUserData	0x00000200
hasMpid	0x00000400
hasMemberGroup	0x00000800
hasLocateBroker	0x00001000
reserved	0x7FFFE000

4.5.2 LimitOrderRejected Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
priceSlideInstruction	1	Enum	Enum PriceSlideInstruction . For additional information, refer to Display Price Slide Instructions documentation .
minQty	4	Int	Number of shares that must be executed to execute any shares.
maxFloorQty	4	Int	Quantity to be displayed at one time with remaining quantity not displayed on the book.
maxReplenishQtyRange	4	Int	Defines the maximum range to be used when calculating a random refresh quantity.
maxReplenishTimeRange	8	Duration	Defines the maximum range to be used when calculating a random refresh time, in nanoseconds.
referencePriceTarget	2	Short	The target relative to the reference base price in basis points of the NBBO spread.
expireTime	8	TimeStamp	Timestamp at which the order's open quantity will be canceled, valid only with GTT time in force. For additional information, refer to GTT documentation .
userData	8	Long	Pass-through field for use by clients, not used by the Exchange.
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or SenderComp.

Field Name	Size	Type	Notes
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

4.6 MarketOrderAccepted

Field Name	Offset	Size	Type	Notes			
messageType	0	1	Byte	'D'/0x44/68			
presenceBits	1	2	Short	See Optional Fields			
transactTime	3	8	TimeStamp				
orderId	11	8	Long				
clOrdId	19	8	Long	Client-specified order identifier that must increase with each new order.			
orderQty	27	4	Int				
marketOrderBitFields	31	2	Bit Field Name		Offset	Size	Notes
			side		0	3	Enum Side.
			isLocateRequired		3	1	True if a locate is required for short-sale, false otherwise.
			timeInForce		4	4	Enum TimeInForce.
			orderCapacity		8	3	Enum OrderCapacity.
			reserved		11	5	Do not use – reserved for the future.
symbolId	33	2	Short				

4.6.1 MarketOrderAccepted Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasSelfMatchScope	0x0001
hasSelfMatchInstruction	0x0002
hasUserData	0x0004
hasMpid	0x0008
hasMemberGroup	0x0010
hasLocateBroker	0x0020
reserved	0xFFC0

4.6.2 MarketOrderAccepted Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .

Field Name	Size	Type	Notes
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
userData	8	Long	Pass-through field for use by clients, not used by the Exchange.
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or Sender-Comp.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

4.7 MarketOrderRejected

The submitted market order has been rejected by the Exchange.

Field Name	Offset	Size	Type	Notes			
messageType	0	1	Byte	'T'/0x54/84			
presenceBits	1	2	Short	See Optional Fields			
transactTime	3	8	TimeStamp				
clOrdId	11	8	Long	Client-specified order identifier that must increase with each new order.			
orderQty	19	4	Int				
marketOrderBitFields	23	2	Bit Field Name		Offset	Size	Notes
			side		0	3	Enum Side .
			isLocateRequired		3	1	True if a locate is required for short-sale, false otherwise.
			timeInForce		4	4	Enum TimeInForce .
			orderCapacity		8	3	Enum OrderCapacity .
			reserved		11	5	Do not use – reserved for the future.
symbolId	25	2	Short				
reason	27	1	Enum	Enum RejectReason . Indicates the reason that a customer action was rejected by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.			

4.7.1 MarketOrderRejected Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasSelfMatchScope	0x0001
hasSelfMatchInstruction	0x0002
hasUserData	0x0004
hasMpid	0x0008
hasMemberGroup	0x0010
hasLocateBroker	0x0020
reserved	0xFFC0

4.7.2 MarketOrderRejected Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
userData	8	Long	Pass-through field for use by clients, not used by the Exchange.
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or Sender-Comp.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

4.8 OrderCanceled

This order has been canceled. No shares are left on the Exchange. The Member will receive this message in response to a CancelOrder message, as well as unsolicited messages from the Exchange for various reasons. See the reason field for details.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'X'/0x58/88
transactTime	1	8	TimeStamp	
orderId	9	8	Long	
origClOrdId	17	8	Long	The ClOrdId of the order to be canceled.
reason	25	1	Enum	Enum CancelReason . Reason that an order was canceled. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.9 CancelRejected

The requested CancelOrder has been rejected. See reason code for details.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'W'/0x57/87
transactTime	1	8	TimeStamp	
origCLOrdId	9	8	Long	The CLOrdId of the order to be canceled.
reason	17	1	Enum	Enum RejectReason . Indicates the reason that a customer action was rejected by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.10 OrderModified

An order has been modified.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'Y'/0x59/89
presenceBits	1	1	Byte	See Optional Fields
transactTime	2	8	TimeStamp	
orderId	10	8	Long	
cLOrdId	18	8	Long	The CLOrdId for this modify message. Must be day-unique.
origCLOrdId	26	8	Long	The CLOrdId of the order to be modified.
leavesQty	34	4	Int	Number of shares that were still available to execute after the modify.

4.10.1 OrderModified Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasOrderQty	0x01
hasModifyBitFields	0x02
hasLocateBroker	0x04
reserved	0xF8

4.10.2 OrderModified Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
orderQty	4	Int	Quantity can remain the same or decrease. Quantity cannot increase. For additional information, refer to Modify Behavior documentation .

Field Name	Size	Type	Notes
modifyBitFields	1	Bit Field Name	Offset
		side	0
		isLocateRequired	3
		reserved	4
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

4.11 ModifyRejected

The requested ModifyOrder has been rejected. See reason code for details.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'N'/0x4e/78
presenceBits	1	1	Byte	See Optional Fields
transactTime	2	8	TimeStamp	
clOrdId	10	8	Long	The ClOrdId for this modify message. Must be day-unique.
origClOrdId	18	8	Long	The ClOrdId of the order to be modified.
reason	26	1	Enum	Enum RejectReason . Indicates the reason that a customer action was rejected by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.11.1 ModifyRejected Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasOrderQty	0x01
hasModifyBitFields	0x02
hasLocateBroker	0x04
reserved	0xF8

4.11.2 ModifyRejected Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
orderQty	4	Int	Quantity can remain the same or decrease. Quantity cannot increase. For additional information, refer to Modify Behavior documentation .
modifyBitFields	1	Bit Field Name	Offset
		Size	Notes
		side	0 3 Enum Side .
		isLocateRequired	3 1 Processed for Sell Short and Sell Short Exempt in which client affirms ability to borrow (isLocateRequired = false) or client does not affirm ability to borrow (isLocateRequired = true)
		reserved	4 4 Do not use – reserved for the future.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

4.12 OrderReplaced

Replace an existing order with a new order.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'J'/0x4a/74
presenceBits	1	2	Short	See Optional Fields
transactTime	3	8	TimeStamp	
orderId	11	8	Long	The new Exchange OrderId of the replace order.
clOrdId	19	8	Long	The ClOrdId for this modify message. Must be day-unique.
origClOrdId	27	8	Long	The ClOrdId of the order to be replaced.

Field Name	Offset	Size	Type	Notes			
replaceBitFields	35	2	Bit Field Name		Offset	Size	Notes
			side		0	3	Enum Side .
			isLocateRequired		3	1	Processed for Sell Short and Sell Short Exempt in which client affirms ability to borrow (isLocateRequired = false) or client does not affirm ability to borrow (isLocateRequired = true)
			isIso		4	1	True for Reg NMS Inter-market sweep orders, false otherwise.
			isPostOnly		5	1	True to only add the order to the book if it would be added as a resting order with no quantity executed.
			cancelAtEntryIfCrossed		6	1	True to cancel the order at entry if market conditions are crossed.
			reserved		7	9	Reserved for future use.
			leavesQty	37	4	Int	Number of shares that were still available to execute after the replace.

4.12.1 OrderReplaced Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasPrice	0x0001
hasOrderQty	0x0002
hasMaxFloorQty	0x0004
hasSelfMatchScope	0x0008
hasSelfMatchInstruction	0x0010
hasPriceSlideInstruction	0x0020
hasReferencePriceTarget	0x0040
hasLocateBroker	0x0080
hasRankPrice	0x0100
hasDisplayPrice	0x0200
reserved	0xFC00

4.12.2 OrderReplaced Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
price	8	Long	

Field Name	Size	Type	Notes
orderQty	4	Int	Quantity can remain the same, decrease or increase. For additional information, refer to Replace Behavior documentation .
maxFloorQty	4	Int	Quantity to be displayed at one time with remaining quantity not displayed on the book.
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
priceSlideInstruction	1	Enum	Enum PriceSlideInstruction . For additional information, refer to Display Price Slide Instructions documentation .
referencePriceTarget	2	Short	The target relative to the reference base price in basis points of the NBBO spread.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.
rankPrice	8	Long	Optional rank price, when different from limit price.
displayPrice	8	Long	Optional display price, when different from limit price.

4.13 ReplaceRejected

The requested ReplaceOrder has been rejected. See reason code for details.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'K'/0x4b/75
presenceBits	1	2	Short	See Optional Fields
transactTime	3	8	TimeStamp	
clOrdId	11	8	Long	The ClOrdId for this modify message. Must be day-unique.
origClOrdId	19	8	Long	The ClOrdId of the order to be replaced.

Field Name	Offset	Size	Type	Notes		
replaceBitFields	27	2	Bit Field Name	Offset	Size	Notes
			side	0	3	Enum Side .
			isLocateRequired	3	1	Processed for Sell Short and Sell Short Exempt in which client affirms ability to borrow (isLocateRequired = false) or client does not affirm ability to borrow (isLocateRequired = true)
			isIso	4	1	True for Reg NMS Inter-market sweep orders, false otherwise.
			isPostOnly	5	1	True to only add the order to the book if it would be added as a resting order with no quantity executed.
			cancelAtEntryIfCrossed	6	1	True to cancel the order at entry if market conditions are crossed.
			reserved	7	9	Reserved for future use.
			reason	29	1	Enum

4.13.1 ReplaceRejected Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasPrice	0x0001
hasOrderQty	0x0002
hasMaxFloorQty	0x0004
hasSelfMatchScope	0x0008
hasSelfMatchInstruction	0x0010
hasPriceSlideInstruction	0x0020
hasReferencePriceTarget	0x0040
hasLocateBroker	0x0080
reserved	0xFF00

4.13.2 ReplaceRejected Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
price	8	Long	
orderQty	4	Int	Quantity can remain the same, decrease or increase. For additional information, refer to Replace Behavior documentation .
maxFloorQty	4	Int	Quantity to be displayed at one time with remaining quantity not displayed on the book.
selfMatchScope	1	Enum	Enum SelfMatchScope . Determines the scope for self-match prevention. For additional information, refer to Self Match Prevention documentation .
selfMatchInstruction	1	Enum	Enum SelfMatchInstruction . Determines the behavior when orders from the same entity would match. For additional information, refer to Self Match Prevention documentation .
priceSlideInstruction	1	Enum	Enum PriceSlideInstruction . For additional information, refer to Display Price Slide Instructions documentation .
referencePriceTarget	2	Short	The target relative to the reference base price in basis points of the NBBO spread.
locateBroker	4	Str(4)	Used for short sale orders to identify the broker that has loaned the stock to settle the short sale. MPID should contain upper-case alpha only.

4.14 OrderExecuted

An order has executed.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'E'/0x45/69
transactTime	1	8	TimeStamp	
orderId	9	8	Long	
clOrdId	17	8	Long	Client-specified order identifier that must increase with each new order.
execPrice	25	8	Long	The price at which these shares were executed.
execId	33	8	Long	
execQty	41	4	Int	Number of shares that were executed.
leavesQty	45	4	Int	Number of shares that were still available to execute after the execution. If this value is 0, then the order has been fully executed.
liquidityIndicator	49	1	Enum	Enum LiquidityIndicator . Liquidity indicator codes for executions. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.15 OrderRestated

Informational message from the Exchange. Sent when the Exchange performs work on the Member's order. The order may have been repriced after entry, or a replenishment may have occurred on a reserve order. For a

replenishment, the OrderId refers to the new displayed portion of the order. For a repricing, the OrderId is the new OrderId for the entire order. The Member may continue to refer to this order using the ClOrdId assigned when entering the order. This message is informational. No action is required by the Member.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'F'/0x46/70
presenceBits	1	1	Byte	See Optional Fields
transactTime	2	8	TimeStamp	
orderId	10	8	Long	A new OrderId has been created, and new priority for this action.
clOrdId	18	8	Long	Client-specified order identifier that must increase with each new order.
reason	26	1	Enum	Enum RestatementReason . Reason for order restatement events triggered by the Exchange.

4.15.1 OrderRestated Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasRankPrice	0x01
hasDisplayPrice	0x02
hasDisplayQty	0x04
reserved	0xF8

4.15.2 OrderRestated Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
rankPrice	8	Long	The rank price. Only present on re-priced orders.
displayPrice	8	Long	The display price, when different from rank price. Only present on re-priced orders.
displayQty	4	Int	The quantity of the displayed replenished reserve order. Only present on reserve orders.

4.16 SelfMatchPrevented

Notification that a self-match was detected and prevented according to the order's self-match prevention instructions. This message is sent when an order would have executed against another order from the same trading entity within the configured self-match scope (BY_MEMBER, BY_MPID, BY_MEMBER_GROUP, or BY_MPID_AND_MEMBER_GROUP). The message reports the quantity that would have executed, the quantity that was canceled, and the remaining quantity still active on the book. For additional information, refer to Self Match Prevention [documentation](#).

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'Z'/0x5a/90
transactTime	1	8	TimeStamp	
orderId	9	8	Long	
clOrdId	17	8	Long	Client-specified order identifier that must increase with each new order.

Field Name	Offset	Size	Type	Notes
execPrice	25	8	Long	The price at which the self-match would have executed.
execId	33	8	Long	
execQty	41	4	Int	The quantity that would have executed against the self-matching order if self-match prevention was not enabled.
canceledQty	45	4	Int	The quantity that was decremented or canceled from this order as a result of the self-match prevention instruction.
leavesQty	49	4	Int	The quantity remaining on the book after the self-match prevention action. If this value is 0, the order has been fully canceled.
liquidityIndicator	53	1	Enum	Enum LiquidityIndicator . Liquidity indicator codes for executions. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.17 MassCancelAccepted

The Mass Cancel has been accepted and is being processed. For additional information, refer to [Mass Cancel documentation](#).

Field Name	Offset	Size	Type	Notes		
messageType	0	1	Byte	'O'/0x4f/79		
presenceBits	1	1	Byte	See Optional Fields		
transactTime	2	8	TimeStamp			
massCancelRequestId	10	8	Long	Unique identifier assigned by the Member for this mass cancel request.		
scope	18	1	Enum	Enum MassCancelScope . Determines the Member-Owned scope for mass cancels. For additional information, refer to Mass Cancel Behavior documentation .		
massCancelBitFields	19	1	Bit Field Name	Offset	Size	Notes
			excludeAuctionOrders	0	1	When set, auction orders are excluded from the mass cancel scope.
			reserved	1	7	Do not use – reserved for the future.
massCancelId	20	8	Long	An identifier assigned by the Exchange for this Mass Cancel request.		

4.17.1 MassCancelAccepted Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasMpid	0x01
hasSenderComp	0x02
hasMemberGroup	0x04
hasClOrdId	0x08
reserved	0xF0

4.17.2 MassCancelAccepted Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
senderComp	8	Str(8)	When present, narrows the cancel to orders entered on the specified order entry port (SenderComp).
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or SenderComp.
clOrdId	8	Long	When present, scopes the mass cancel to this specific order. SenderComp is required when ClOrdId is supplied.

4.18 MassCancelRejected

The Mass Cancel has been rejected.

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'P'/0x50/80
presenceBits	1	1	Byte	See Optional Fields
transactTime	2	8	TimeStamp	
massCancelRequestId	10	8	Long	Unique identifier assigned by the Member for this mass cancel request.
scope	18	1	Enum	Enum MassCancelScope . Determines the Member-Owned scope for mass cancels. For additional information, refer to Mass Cancel Behavior documentation .
massCancelBitFields	19	1	Bit Field Name	Offset
			excludeAuctionOrders	0
			reserved	1
			Size	Notes
			1	When set, auction orders are excluded from the mass cancel scope.
			7	Do not use – reserved for the future.

Field Name	Offset	Size	Type	Notes
reason	20	1	Enum	Enum RejectReason . Indicates the reason that a customer action was rejected by the Exchange. Additional values may be introduced in future Exchange releases without changing the message layout. Members should be prepared to receive values not listed here, and should treat any such value as a generic, unspecified member of this set rather than rejecting the message or ending the session.

4.18.1 MassCancelRejected Presence Bits

These presence bits are as follows. Any added optional fields must be in their presence bit order [See Optional Fields](#)

Bit Name	Set Mask
hasMpid	0x01
hasSenderComp	0x02
hasMemberGroup	0x04
hasClOrdId	0x08
reserved	0xF0

4.18.2 MassCancelRejected Optional Fields

The optional fields begin directly after the end of the field above. If a bit is set in the presence bits, the field must be present, and in the order listed in the presence bits. [See Optional Fields](#)

Field Name	Size	Type	Notes
mpid	4	Str(4)	Market Participant Identifier. MPID should contain upper-case alpha only.
senderComp	8	Str(8)	When present, narrows the cancel to orders entered on the specified order entry port (SenderComp).
memberGroup	2	Str(2)	Member supplied grouping. Used for both self-match prevention and mass cancel, in conjunction with MPID or SenderComp.
clOrdId	8	Long	When present, scopes the mass cancel to this specific order. SenderComp is required when ClOrdId is supplied.

4.19 MassCancelResult

Confirms sweep completion. Sent after all affected orders have been canceled. For additional information, refer to Mass Cancel [documentation](#).

Field Name	Offset	Size	Type	Notes
messageType	0	1	Byte	'Q'/0x51/81
transactTime	1	8	TimeStamp	
massCancelRequestId	9	8	Long	The MassCancelRequestId assigned by the Member.
massCancelId	17	8	Long	The MassCancelId assigned by the Exchange, found in the MassCancelAccepted.
canceledCount	25	4	Int	The number of orders canceled from the request.